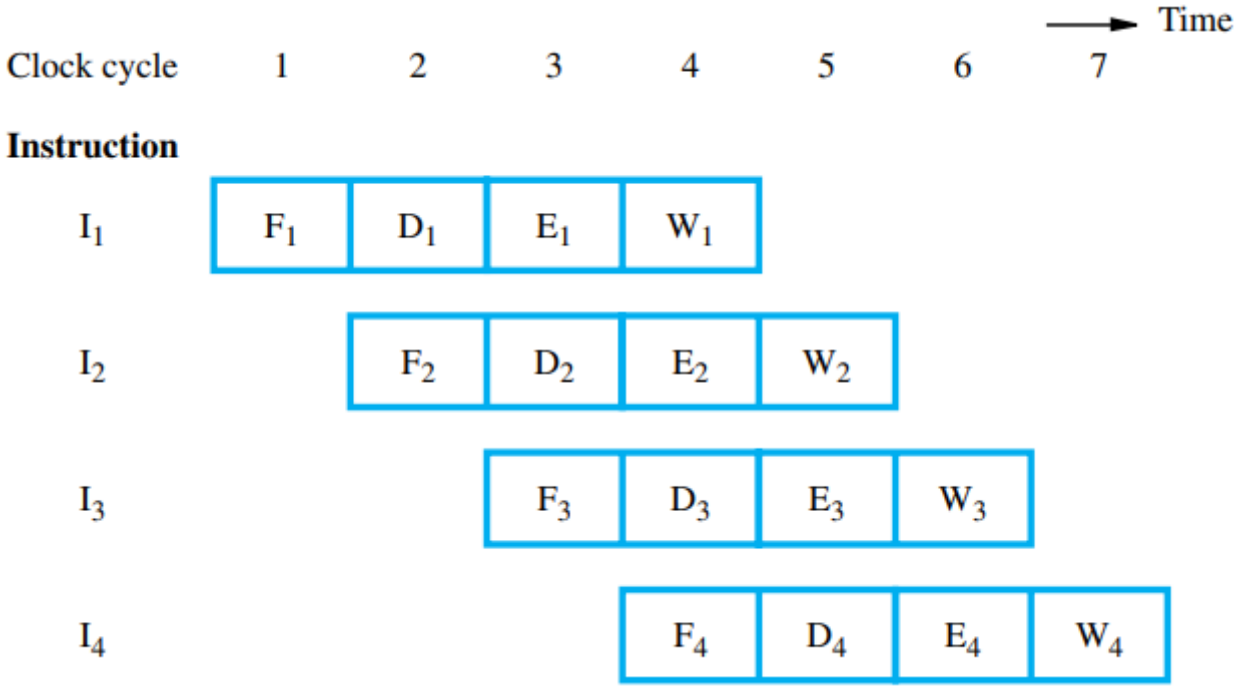
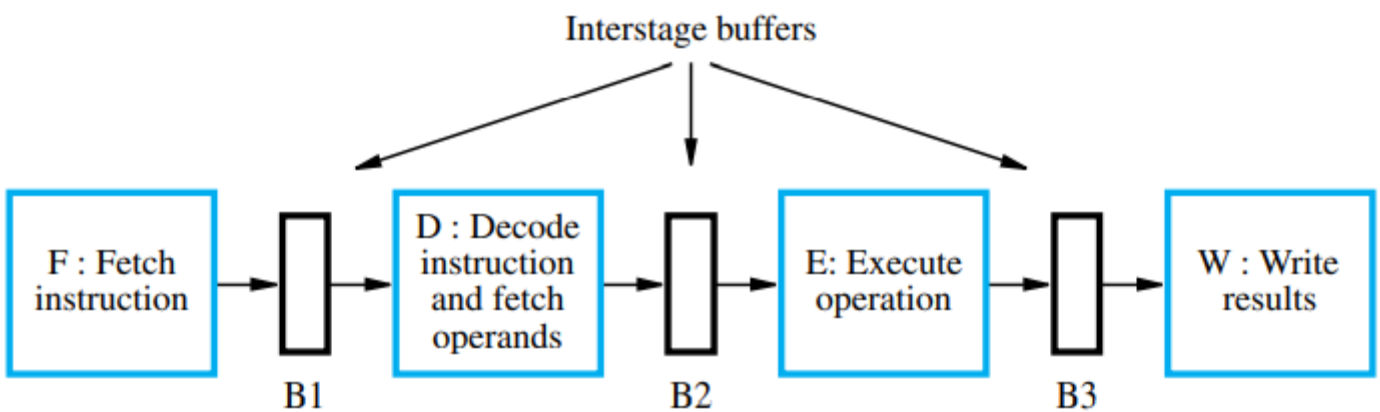
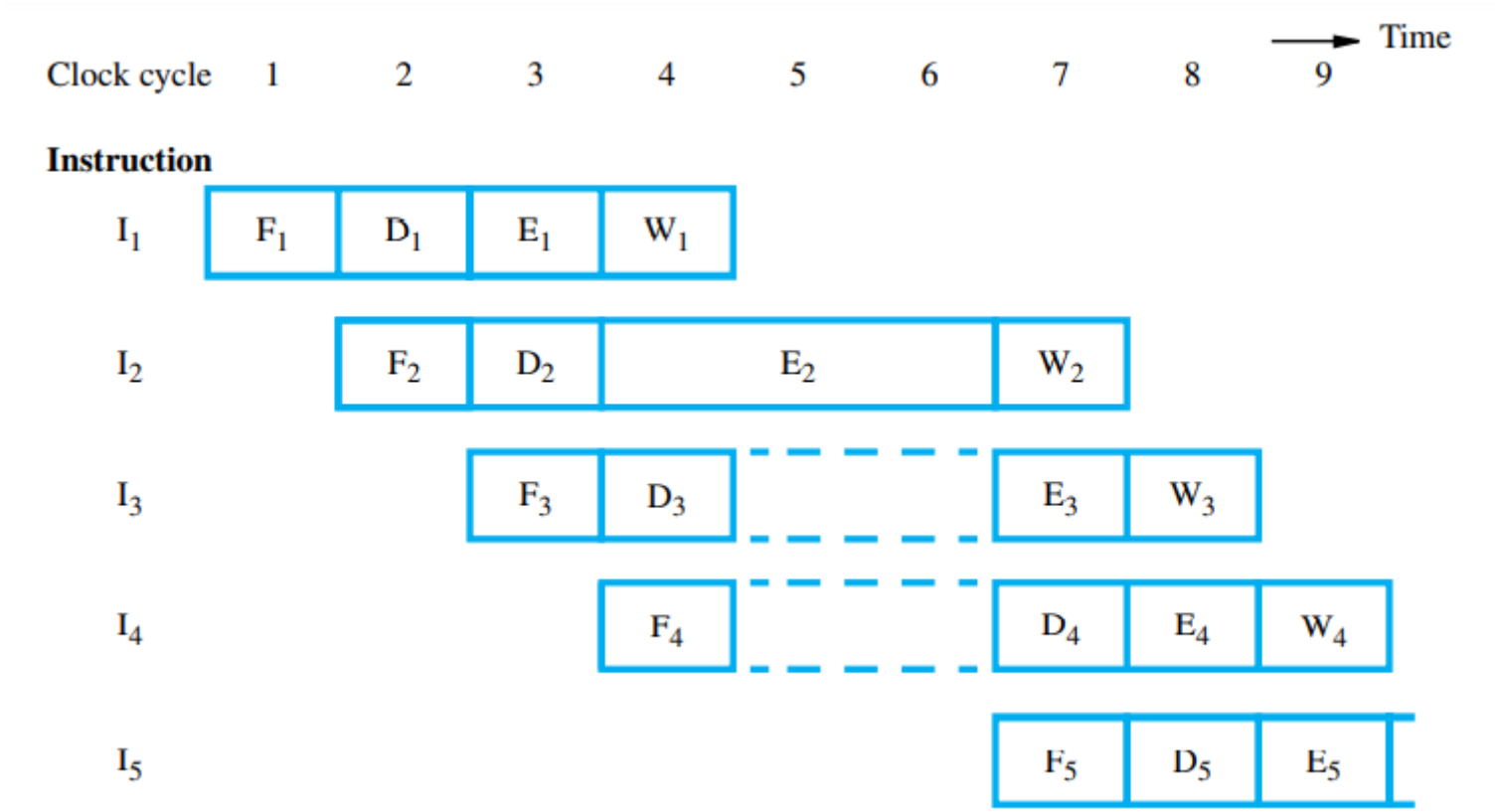
# Pipelining

# Pipelining Basics



# Stall in Pipeline



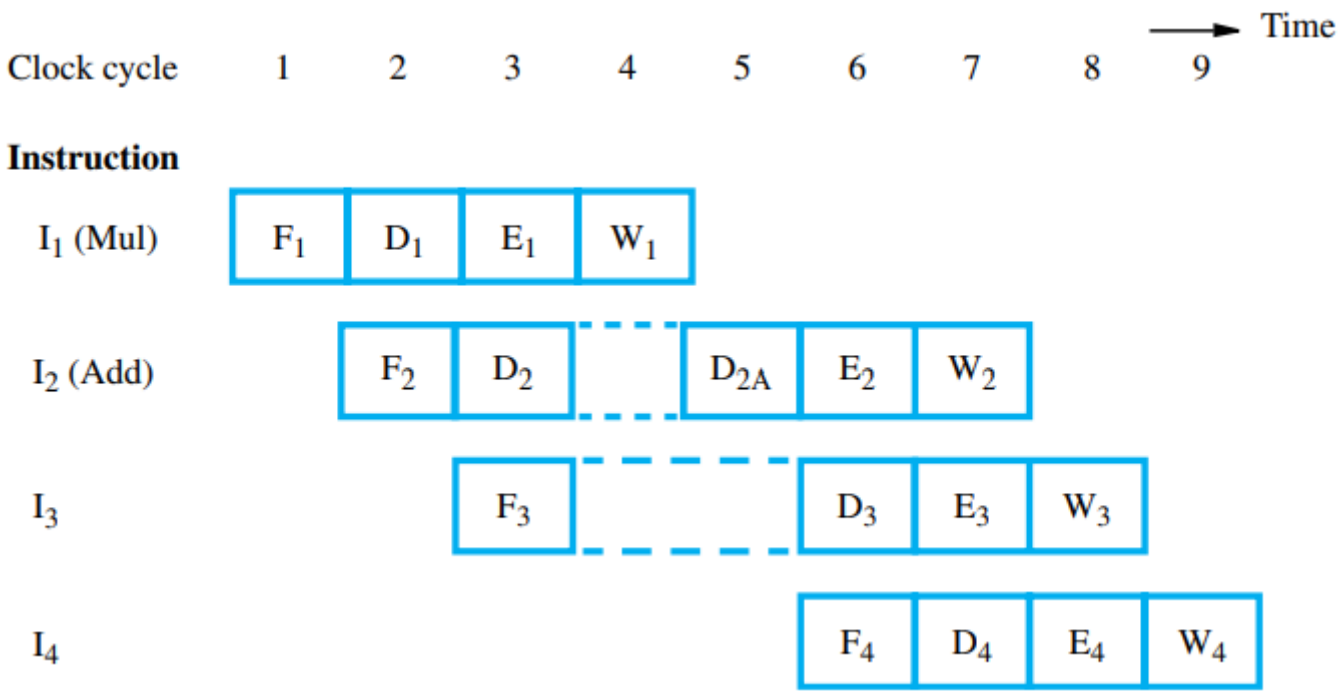
# Pipelining Hazards

- Data Hazard (RAW, WAW, WAR Hazard)
- Instruction Hazard / Control Hazard
- Structural Hazard

# Data Hazard

**A <- 3 x A**

**B <- 4 + A**



# Read After Write (RAW) Hazard

- Later instruction tries to read an operand before earlier instruction writes it.

- add R1, R2, R3
  sub R5, R1, R4

- RAW hazard (also known as true dependency) is extremely common.

# Write After Read (WAR) Hazard

- Later instruction tries to write an operand before earlier instruction reads it.

- add R1, R2, R3
  sub R2, R5, R4

- WAR hazard (also known as anti dependency) is extremely uncommon.

# Write After Write (WAW) Hazard

- Later instruction tries to write an operand before earlier instruction writes it.

- add R1, R2, R3
  sub R1, R2, R4

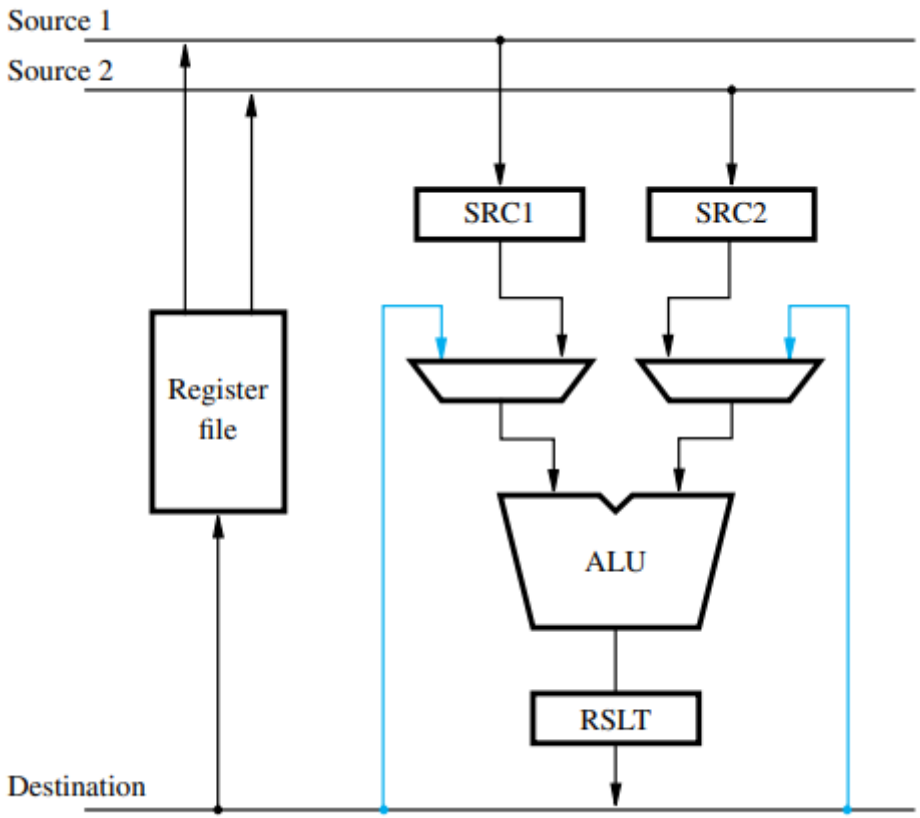
- WAW hazard (also known as output dependency) is possible in a reasonable pipeline.

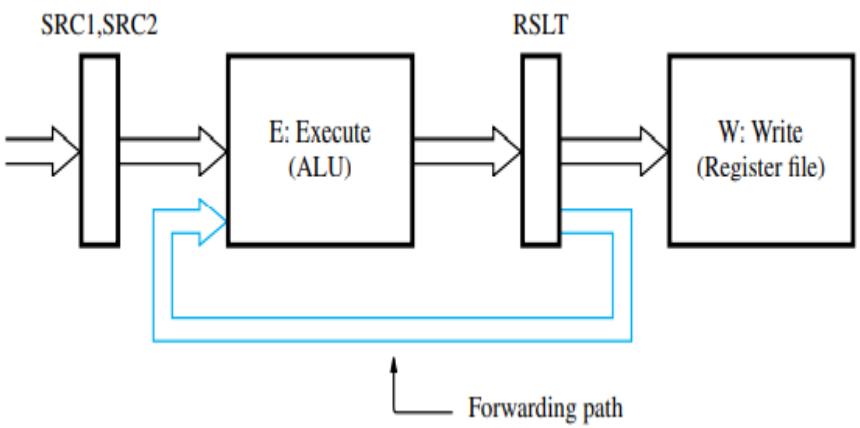
- RAR is not a hazard.

# Solution to Data Hazard

- Insert a pipeline bubble whenever a read after write (RAW) dependency is encountered, guaranteed to increase latency.
- Use out-of-order execution to potentially prevent the need for pipeline bubbles.
- Use operand forwarding.
- Use register renaming.

# Operand Forwarding



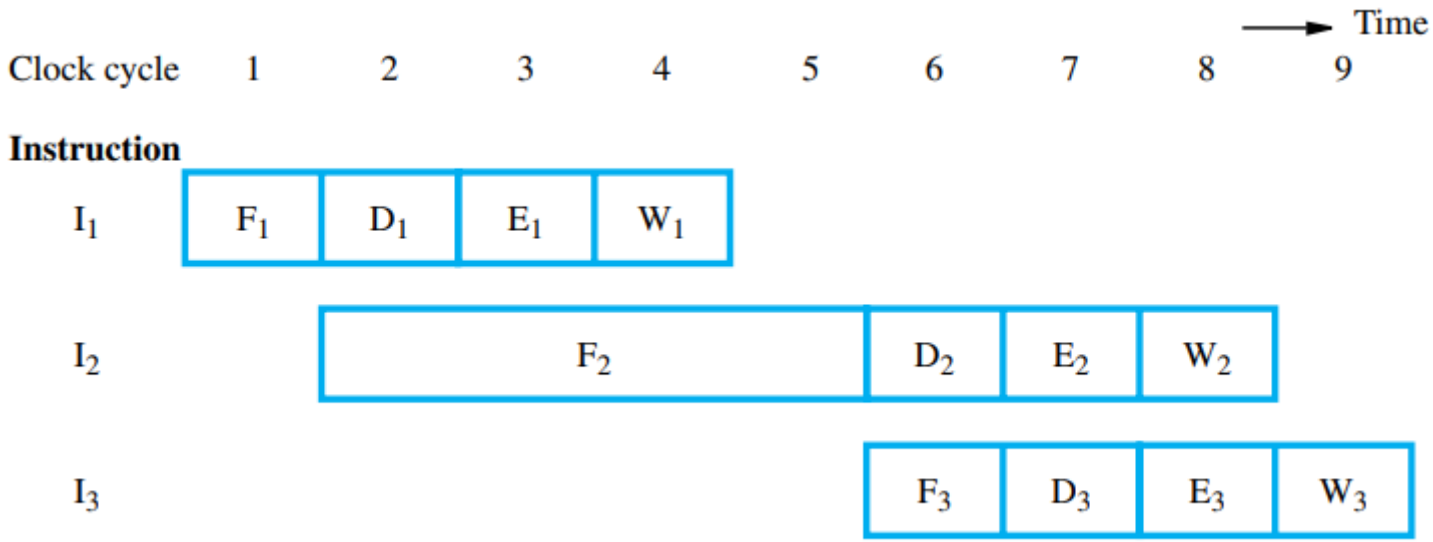



# Software Approach to Handle Data Hazard

```
I1:  Mul   R2,R3,R4
     NOP
     NOP
I2:  Add   R5,R4,R6
```

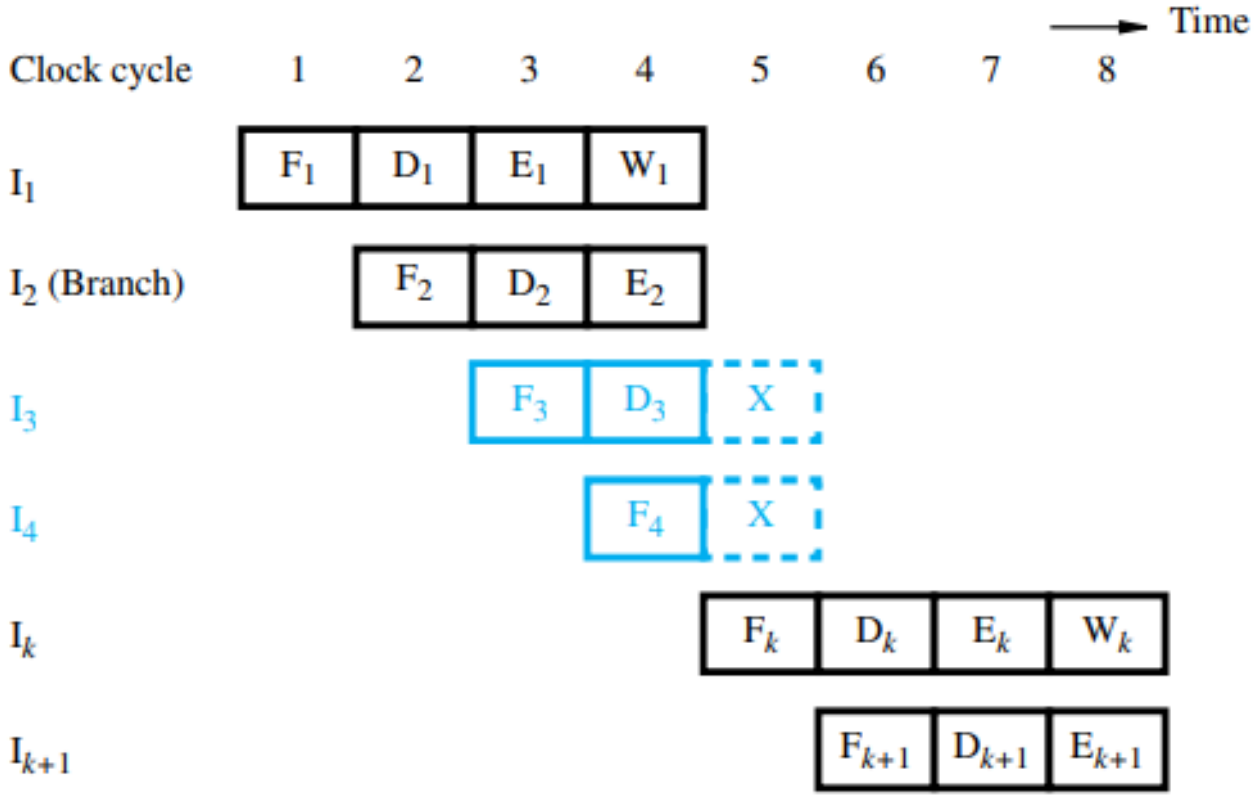
# Instruction Hazard

**Cache Miss**



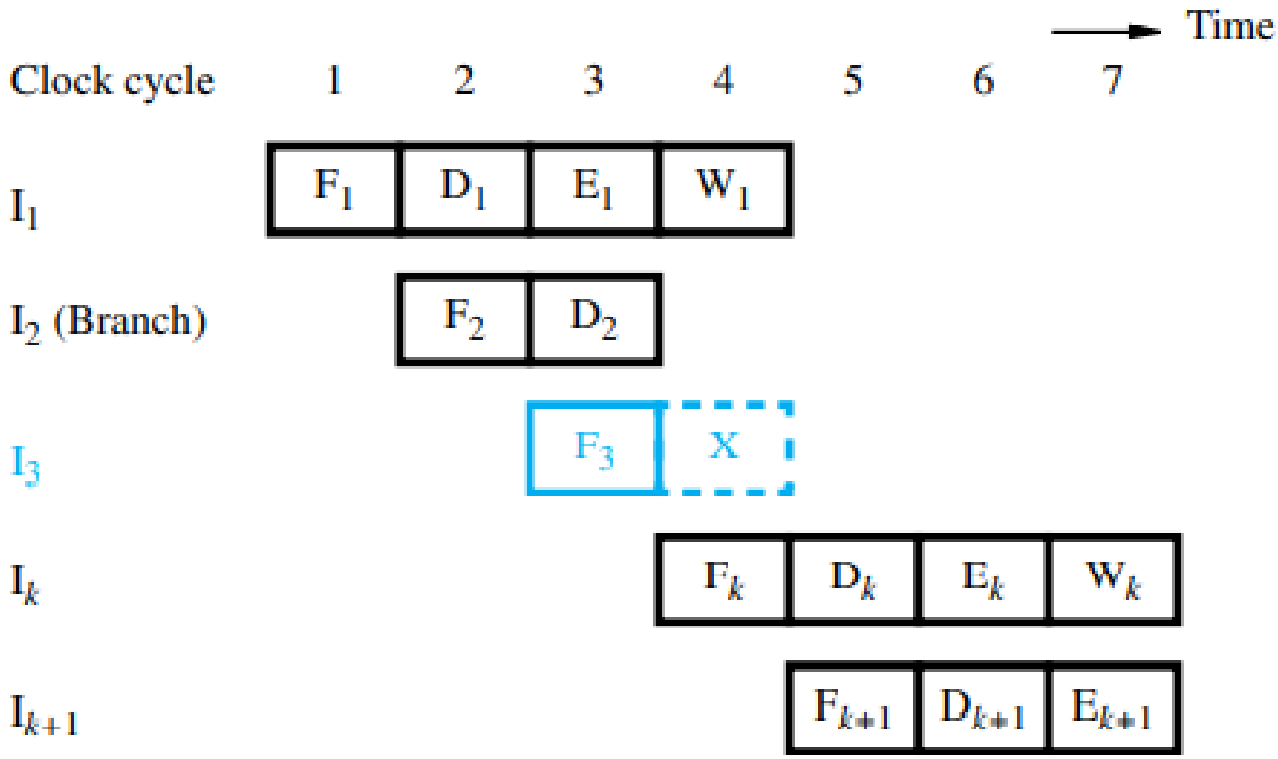
# Instruction Hazard (Unconditional Branch)

**Branch address computed during Execute stage**

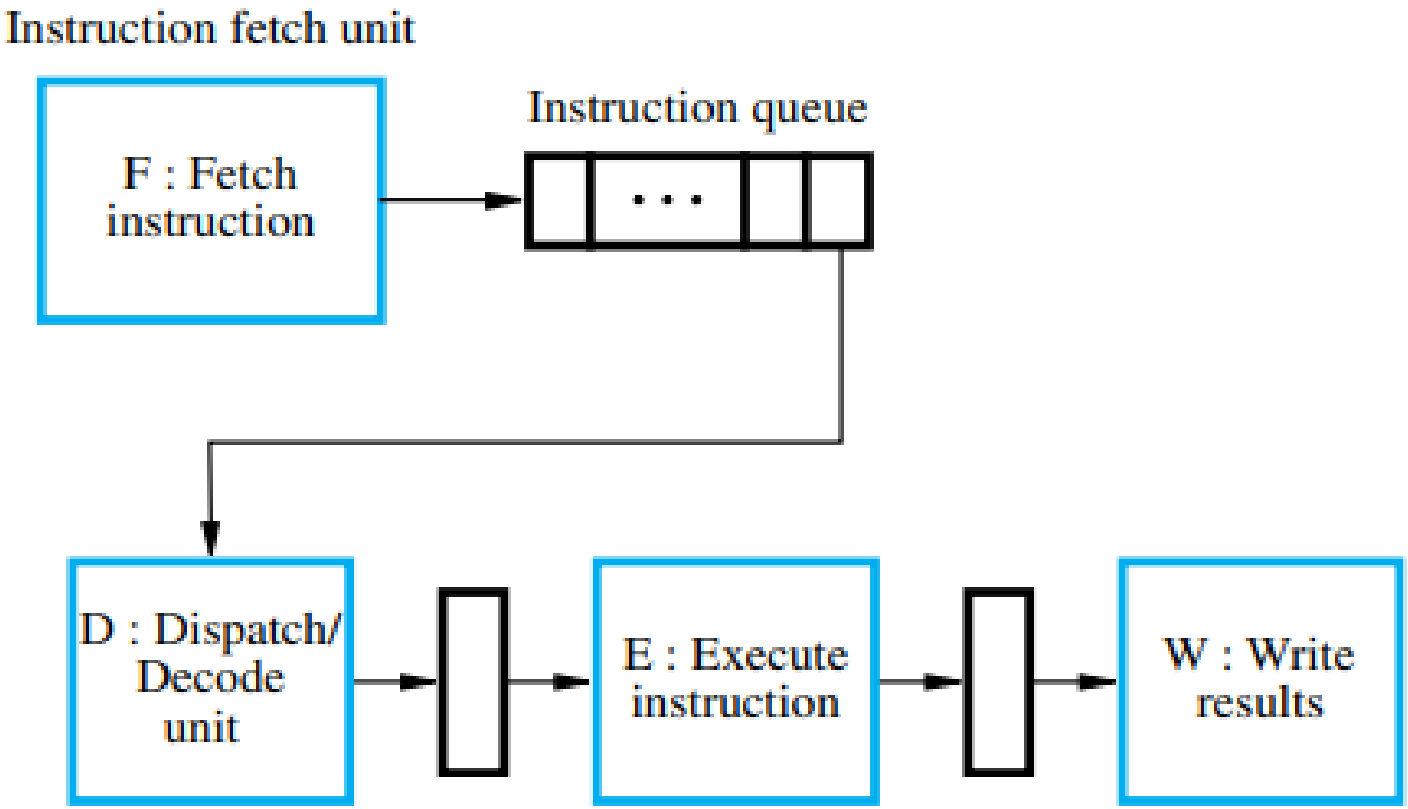


# Instruction Hazard (Unconditional Branch)

**Branch address computed during Decode stage**

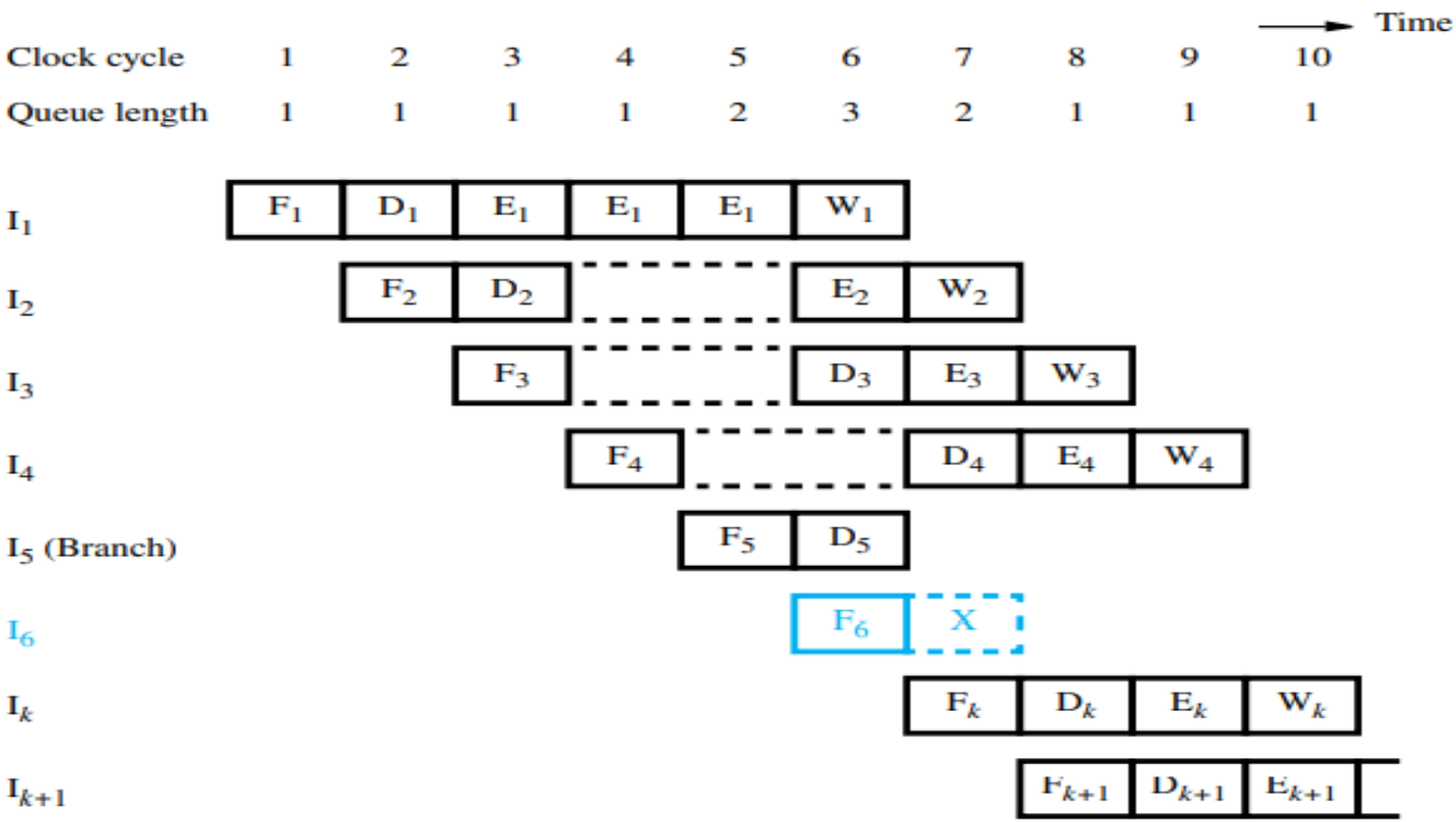


# Instruction Queue and Prefetching



# Instruction Queue and Prefetching

**The branch instruction would normally cause a stall in cycle 7 as a result of discarding instruction I6. Instead, instruction I4 is dispatched from the queue to the decoding stage.**



# Instruction Hazard (Conditional Branch)

- It introduces the added hazard caused by the dependency of the branch condition on the result of a preceding instruction. The decision to branch cannot be made until the execution of that instruction has been completed.

- **Delayed Branch**
- **Branch Prediction (Static & Dynamic Branch Prediction)**
- **Dynamic Branch Prediction (1-bit & 2-bit Branch Prediction)**

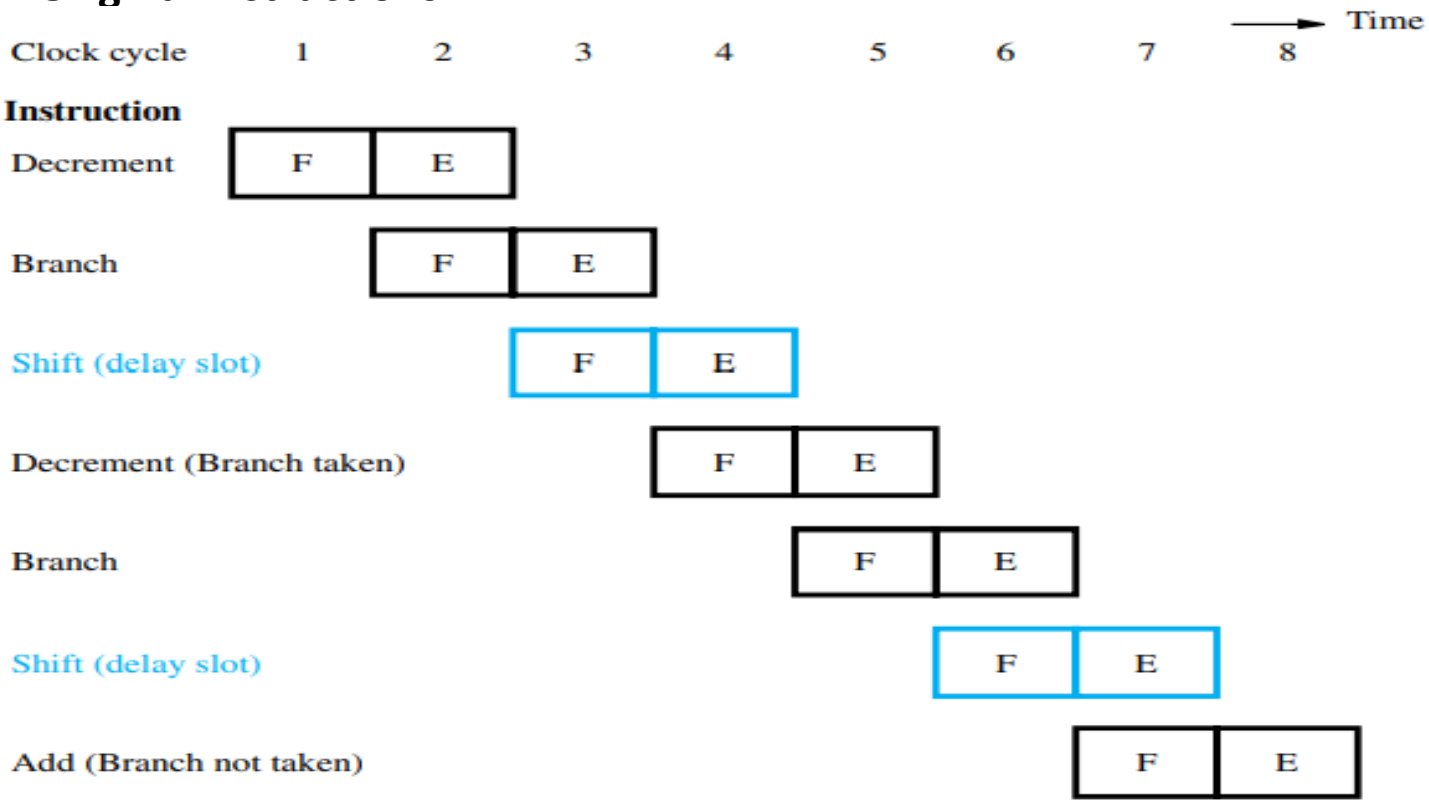
# Delayed Branch

**Original Instructions**

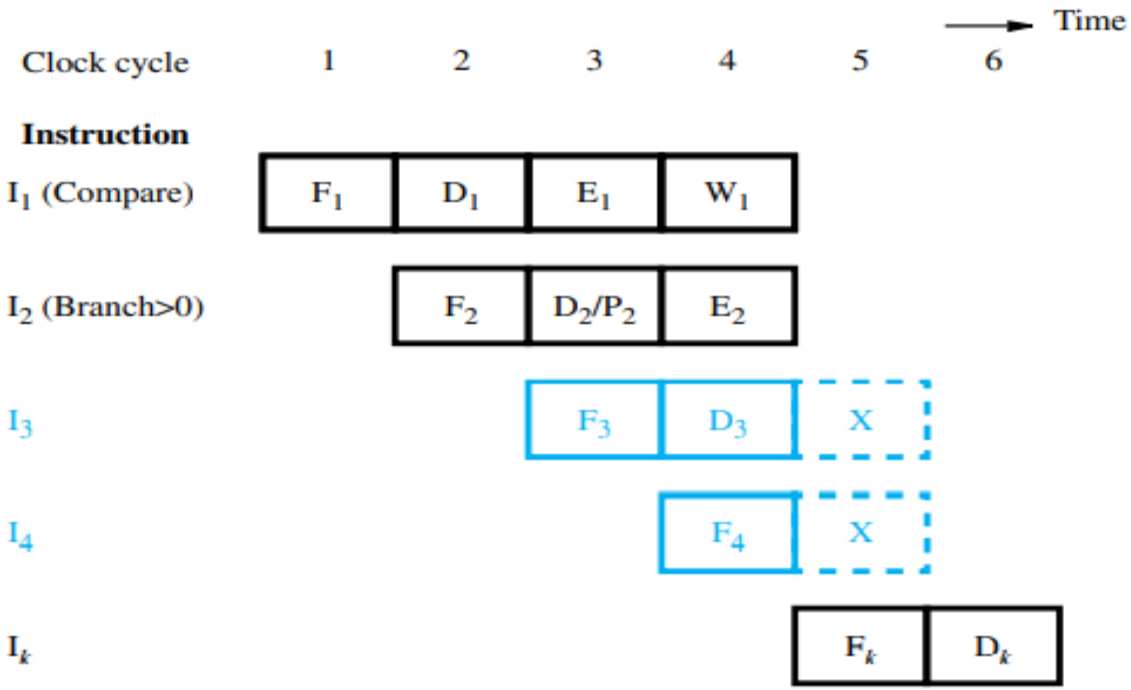
| | | |
|---|---|---|
| LOOP | Shift_left | R1 |
| | Decrement | R2 |
| | Branch=0 | LOOP |
| NEXT | Add | R1,R3 |

**Reordered Instructions**

| | | |
|---|---|---|
| LOOP | Decrement | R2 |
| | Branch=0 | LOOP |
| | Shift_left | R1 |
| NEXT | Add | R1,R3 |



# Branch Prediction



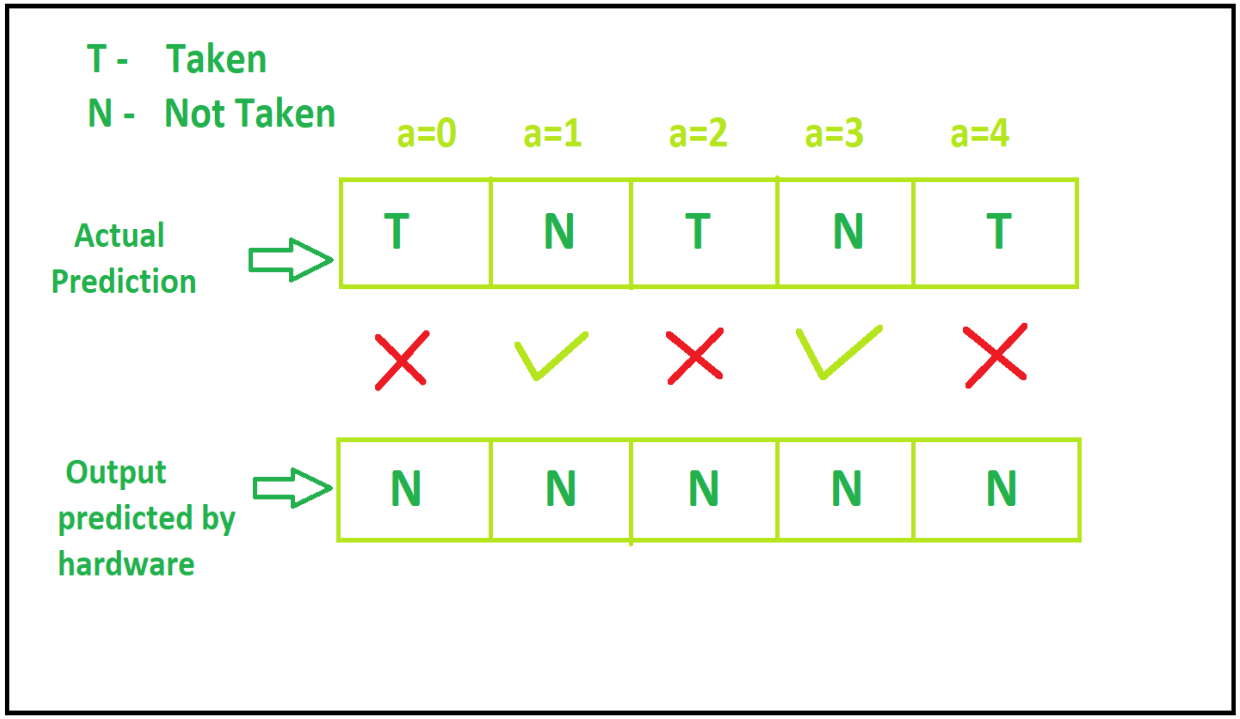

**Effect of a branch instruction incorrectly predicted**

- **Branch Prediction (Static & Dynamic Branch Prediction)**

# Static Branch Prediction

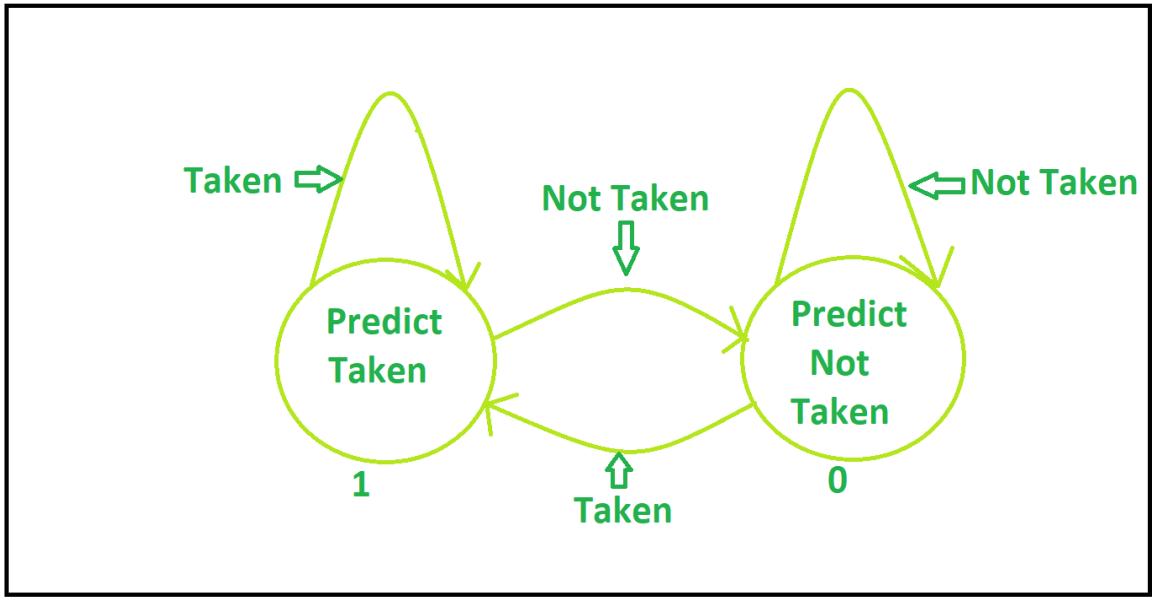
- In static branch prediction technique underlying hardware assumes that either the branch is not taken always or the branch is taken always.
- //Code

```
int a=0;
while(a<5) {
//branch instruction, condition either true or false
if(a%2==0)
{.....}
a++;
}
```



# Dynamic Branch Prediction

- In Dynamic branch prediction technique prediction by underlying hardware is not fixed, rather it changes dynamically. This technique has higher accuracy than static technique.
- **1-bit branch prediction:** In this technique hardware changes its assumption just after one false assumption. For example, if hardware assumes branch to be taken but actually branch is not taken, then in next step hardware assumes branch to be not taken and vice-versa.



# 1-Bit Branch Prediction

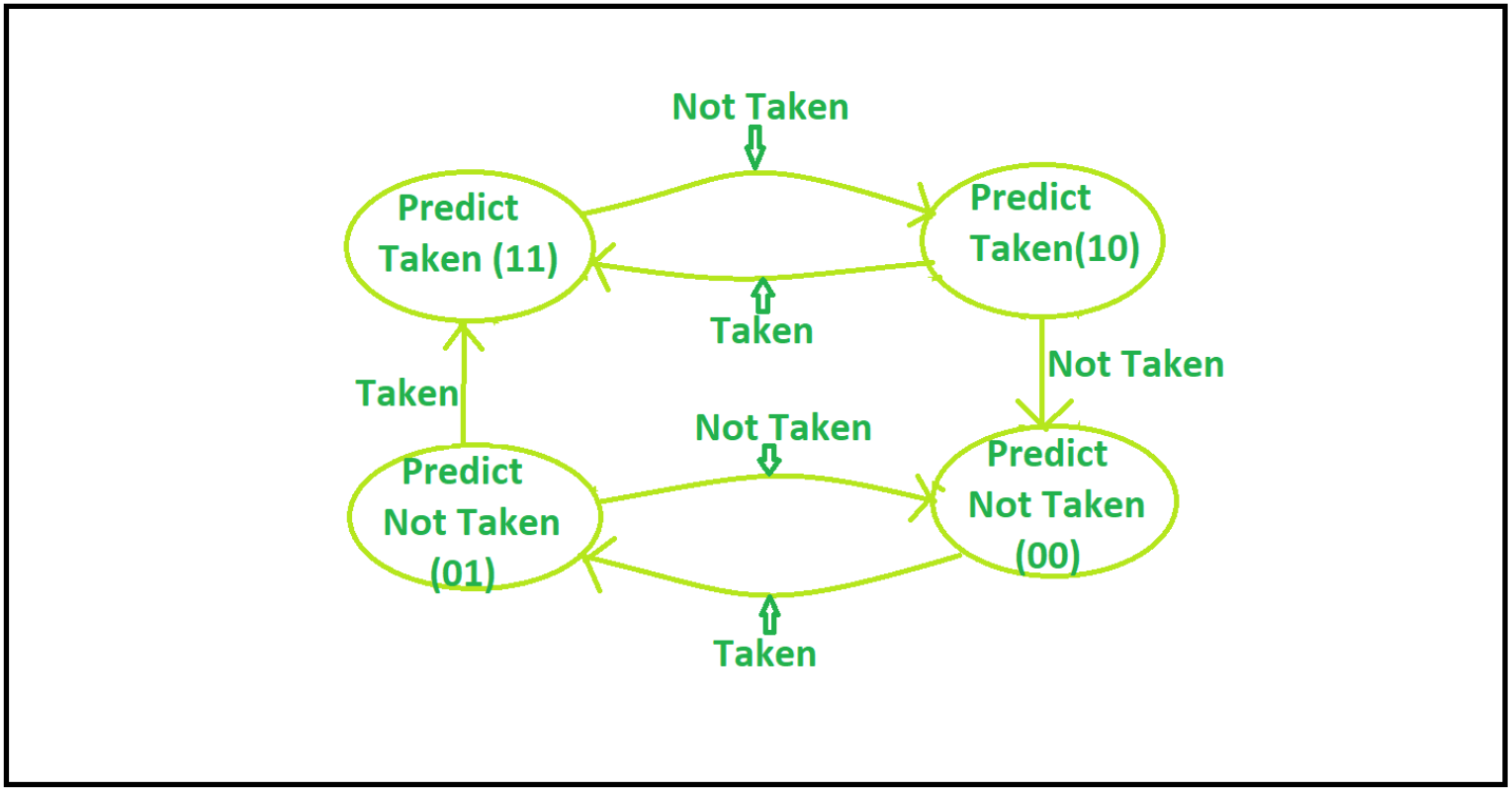
At the beginning, let hardware assume branch to be taken and so at a=0, branch is taken. At a=1, hardware assumes branch to be taken but branch is not taken. So now at a=2 hardware assumes branch not to be taken but branch is taken. At a=3 hardware assumes branch to be taken but branch is not taken. At a=4 hardware assumes branch not to be taken but branch is taken.

T - Taken
N - Not Taken

| | a=0 | a=1 | a=2 | a=3 | a=4 |
|---|---|---|---|---|---|
| Actual Prediction | T | N | T | N | T |
| | ✓ | ✗ | ✗ | ✗ | ✗ |
| Output predicted by hardware | T | T | N | T | N |

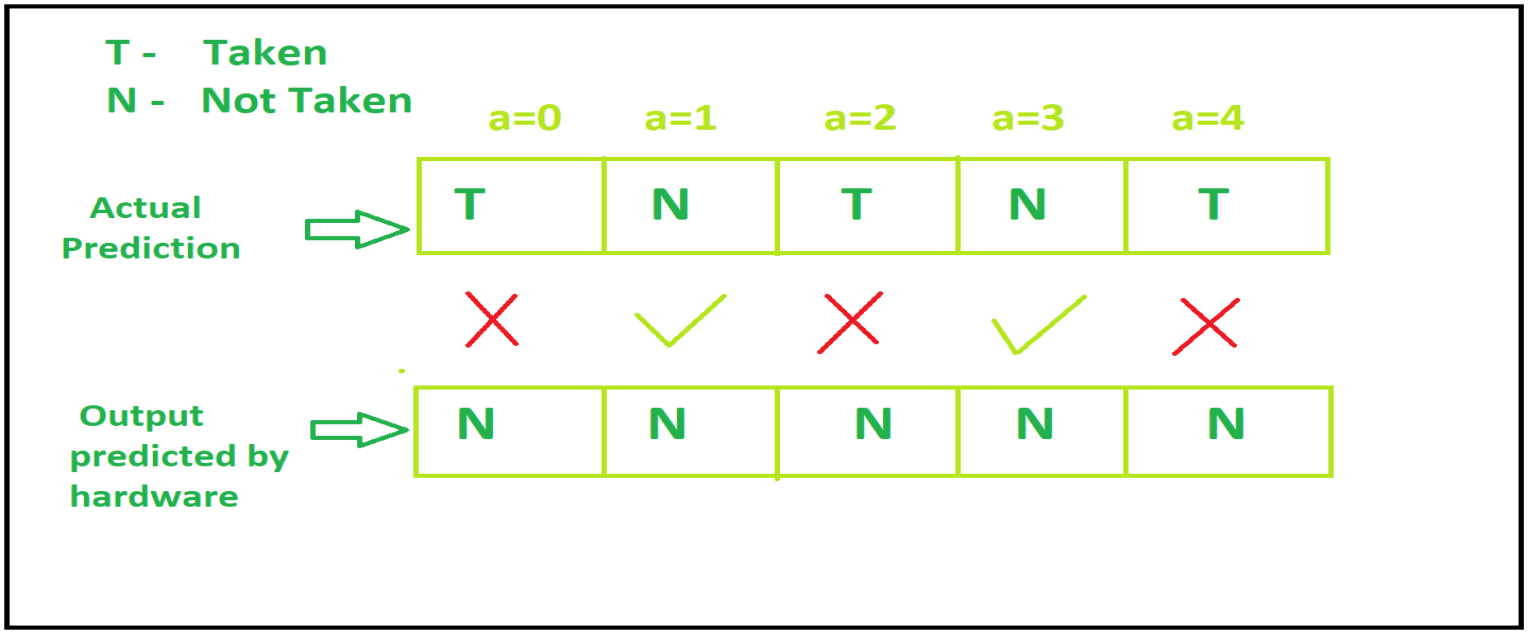
# 2-Bit Branch Prediction

In this technique, the underlying hardware does not changes its assumption just after one incorrect assumption, rather it changes its assumption after two consecutive wrong assumption and vice-versa.



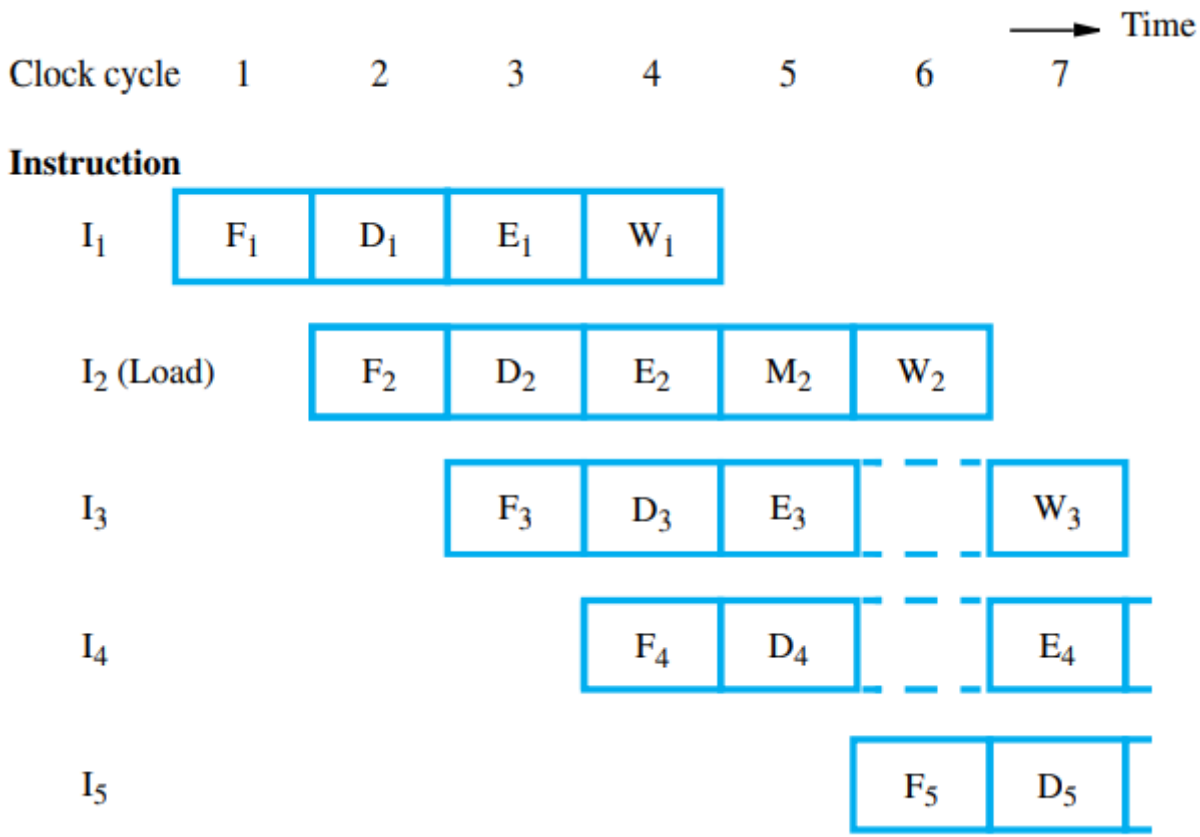
# 2-Bit Branch Prediction

1. Lets say when a=0 everything is reset(00) and so hardware assume branch not to be taken and branch is taken. So current state is (01)
2. When a=1, hardware assumes branch not to be taken and branch is not taken. So current state is (00)
3. When a=2, hardware assumes branch not to be taken and branch is taken. So current State is (01)
4. When a=3, hardware assumes branch not to be taken and branch is not taken. So current State is (10)
5. When a=4, hardware assumes branch not to be taken and branch is taken. So current State is(00)



# Structural Hazard

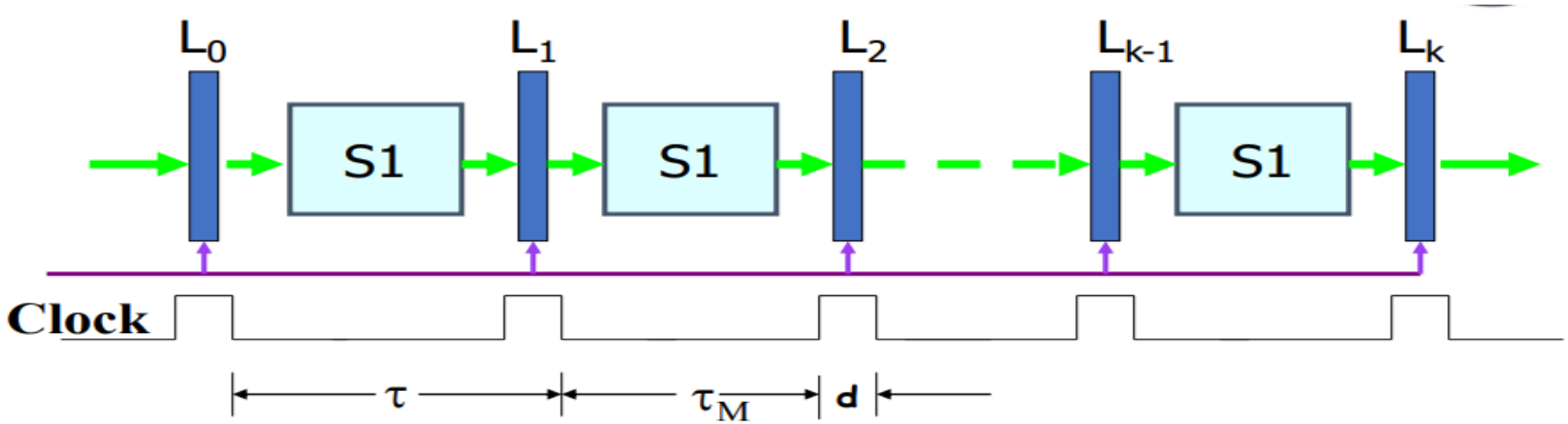
**Load X(R1),R2**



# Linear Pipelining

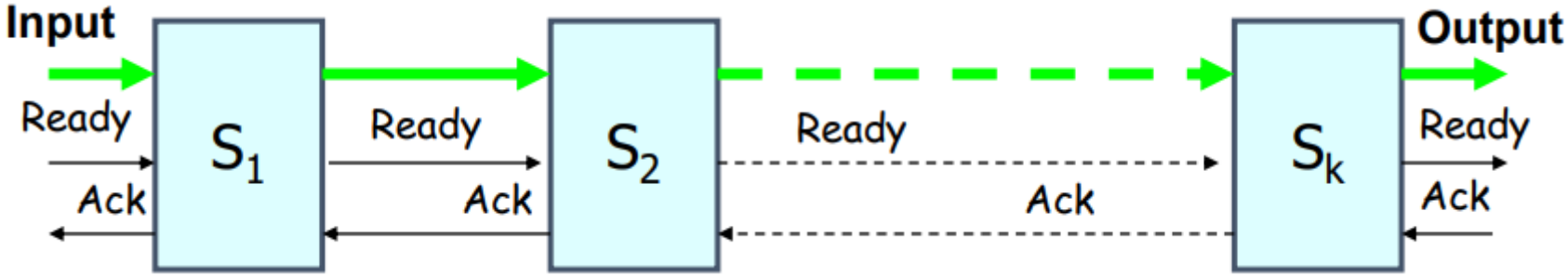
- A linear pipeline processor is a cascade of processing stages which are linearly connected to perform fixed function over a stream of data flowing from one end to the other.
- Linear pipeline are static pipeline because they are used to perform fixed functions.
- On the basis of the control of data flow along the pipeline. we model linear pipelines in two categories:
  - **Synchronous Pipeline**
    - Clocked latches between Stage i and Stage i+1
    - Equal delays in all stages
  - **Asynchronous Pipeline (Handshaking)**

# Synchronous Pipeline




- Clocked latches are used to interface between stages. Latches are used for holding immediate results between stages. On the arrival of a clock pulse, all latches transfer data to the next stage simultaneously.
- The pipeline stages are combinational logic circuits. It is desired to have approximately equal delays in all stages.
- These delays determine the clock period and thus the speed of the pipeline.

# Asynchronous Pipeline




- Data flow between adjacent stages in an asynchronous pipeline is controlled by a handshaking protocol.
- Asynchronous pipelines are useful in designing communication channels in message passing multicomputer.
- Asynchronous pipelines may have variable throughput rate. Different amount of delay may be experienced in different stages.

# Clock Period & Frequency

- Consider $t_i$ to be time delay due to logic circuitry in stage $S_i$ , d to be time delay of each interface latch.
- Then **clock period**, t , of a linear pipeline is given by
  $t = \max \{t_i\} + d = t_M + d$
- **Frequency**, $f = 1 / t = 1 / [t_M + d]$ (i.e., reciprocal of clock period)

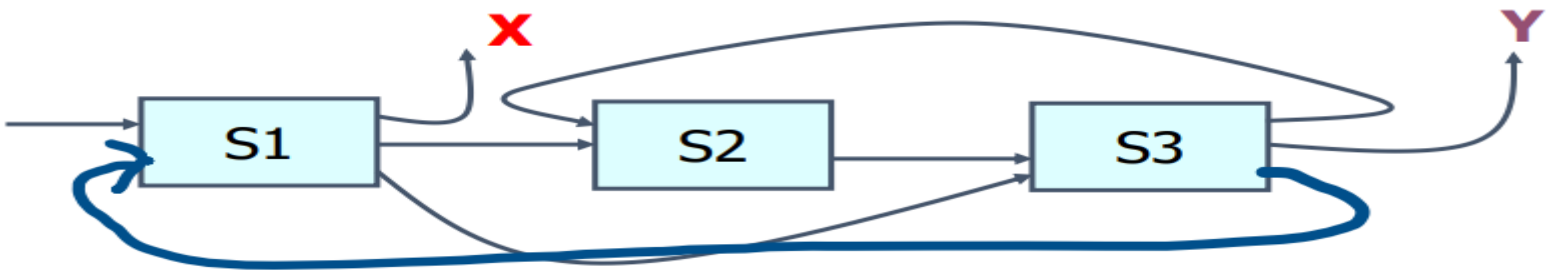
# SpeedUp, Efficiency & Throughput

- Ideally, a linear pipeline of k stages can process n task in k + (n-1) clock cycle, so total time required is $T_K = \{ k + (n-1)\} t$
- Time required for non-pipelined processing $T_1 = nkt$
- **Speedup** $S_K = T_1 / T_K$
  $= nk / k + (n-1)$
- The maximum speedup is $S_K \rightarrow k$ as $n \rightarrow \infty$.
- This maximum speedup is very difficult to achieve because of the dependence structure of the program.

- **Pipeline efficiency** : $\eta = S_K / k$
  $= n / k + (n-1)$

- **Throughput:** Number of task performed per unit time
  $H_k = n / \{ k + (n-1)\} t$
  $= n f / k +(n-1)$ [ f: frequency]
  $= \eta / t$ (Efficiency / t)

# Non-Linear Pipeline




- Non-Linear pipeline are dynamic pipeline because they can be reconfigured to perform variable functions at different times.
- Non-Linear pipeline allows feed-forward and feedback connections in addition to the streamline connection.
- **Example:** Serving one single queue of customers in a bank with two cash counters
- **Self-Study:** Difference between Linear and Non-Linear Pipeline

# Classification of Pipeline Processors

- According to the **levels of processing**

  Arithmetic pipelining

  Instruction pipelining

  Processor pipelining

- According to pipeline **configuration and control strategy**

  Unifunction and Multifunction pipeline

  Static and Dynamic pipeline

  Scalar and Vector pipeline

# Arithmetic, Instruction and Processor pipelining

- **Arithmetic Pipelining:** ALU of a computer can be segmented for pipeline operations in various data formats
- **Instruction Pipelining:** Execution of a stream of instructions can be pipelined by overlapping the execution of the current instruction with fetch, decode etc. of subsequent instructions.
- **Processor Pipelining:** It refers to pipeline processing of the same data stream by a cascade of processors, each of which processes a specific task. The data stream passes the first processor with results stored in a memory block which is also accessible by the second processor. The second processor then passes the refined results to the third and so on.

# Unifunction and Multifunction Pipeline

- A pipeline unit with a fixed and dedicated function is known as **unifunction pipeline. Ex:** Floating point adder

- A **multifunctional pipeline** unit may perform different functions either at different times or at the same time by interconnecting different subsets of stages in the pipeline. **Ex:** Variety of ALU operations

# Static and Dynamic Pipeline

- A **static pipeline** assumes only one functional configuration at a time. Static pipeline can be either unifunctional or multifunctional. A unifunctional pipeline must be static.

- A **dynamic pipeline** processor permits several functional configurations to exist simultaneously. So, a dynamic pipeline must be multifunctional.

# Scalar and Vector Pipeline

- **Scalar pipeline** processes a sequence of scalar operands under the control of a loop. Instructions in a small loop are prefetched into instruction buffer. Required scalar operands for repeated scalar instructions are moved into a data cache in order to continuously supply the pipeline with operands.

- **Vector pipelines** are specifically designed to handle vector instructions over vector operands.